Chapter 5: Client-Side Programming -JavaScript

Client-Side Programming -JavaScript

- **Client-side scripting:-** generally refers to the class of computer programs on the web that are executed client-side, by the user's web browser, instead of server-side (on the web server).
- When the World Wide Web first became popular, ***Hypertext Markup Language (HTML) was the only language to create Web pages.***
- HTML is not a programming language, but a "markup" language and ***has many limitations.***
  - HTML positions text and graphics on a Web page but offers limited interactivity within the Web page.

Client-Side Scripting Using JavaScript

JavaScript is:

- JavaScript is the programming language of the web.
- Developed by Netscape Corporation.
- Is a scripting language a scripting language is lightweight programming language.
- JavaScript interpreted programming language (scripts execute without preliminary compilation) .
- Designed to add interactivity to HTML document.
- Whether used for client-side or server-side solutions, it allows programmers to add interactivity to Web pages without using server-based applications.

What is JavaScript?

- You can use JavaScript for **client side** applications such as
  - Validate specific form fields of data to make sure the client has entered the required fields.
  - Begin processing client information before it reaches the more resource-intensive server processes.
  - It can calculate, manipulate and validate data.
  - Changing content and styles in modern browsers dynamically(It can update and change both HTML and CSS.).
    - **JavaScript Can Change HTML Content**
      - One of many JavaScript HTML methods is getElementById()
    - **JavaScript Can Change HTML Attribute Values**
    - **JavaScript Can Change HTML Styles (CSS)**

What is JavaScript?

JavaScript Can Change HTML Content

- The example below "finds" an HTML element (with id="demo"), and changes the element content (innerHTML) to "Hello JavaScript":

```
<!DOCTYPE html>
<html>
<body>
<h2>What Can JavaScript Do?</h2>
<p id="demo">JavaScript can change HTML content.</p>
<button type="button" onclick='document.getElementById("demo").innerHTML
= "Hello JavaScript!"'>Click Me!</button>
</body>
</html>
```

What is JavaScript?

JavaScript Can Change HTML Attribute Values

- In this example JavaScript changes the value of the src (source) attribute of an

```
<!DOCTYPE html>
<html>
<body>
<h2>What Can JavaScript Do?</h2>
<p>JavaScript can change HTML attribute values.</p>
<p>In this case JavaScript changes the value of the src (source) attribute of an image.</p>
<button onclick="document.getElementById('myImage').src='pic_bulbon.gif'">Turn on the ight</button>
<img id="myImage" src="pic_bulboff.gif" style="width:100px">
<button onclick="document.getElementById('myImage').src='pic_bulboff.gif'">Turn off the light</button>
</body>
</html>
```

What is JavaScript?

JavaScript Can Change HTML Styles (CSS)

```
<!DOCTYPE html>
<html>
<body>
<h2>What Can JavaScript Do?</h2>
<p id="demo">JavaScript can change the style of an HTML element.</p>
<button type="button" onclick="document.getElementById('demo').style.fontSize='35px'">Click Me!</button>
</body>
</html>
```

What is JavaScript?

JavaScript Can Hide HTML Elements

- Hiding HTML elements can be done by changing the display style:

```
<!DOCTYPE html>
<html>
<body>
<h2>What Can JavaScript Do?</h2>
<p id="demo">JavaScript can hide HTML elements.</p>
<button type="button"
onclick="document.getElementById('demo').style.display='none'
">Click Me!</button>
</body>
</html>
```

Why Study JavaScript?

- JavaScript is one of the 3 languages all web developers must learn:
  1. **HTML** to define the content of web pages
  2. **CSS** to specify the layout of web pages
  3. **JavaScript** to program the behavior of web pages

Characteristics of JavaScript

- **JavaScript is a scripting language**
  - A scripting language's like JavaScript are a simple programming language designed to enable computer users to write useful programs easily.
- **JavaScript is both object-based and object-oriented**
  - An **object-based language** means it primarily deals with objects, but does not necessarily support classical OOP features like inheritance, encapsulation, and polymorphism.
  - JavaScript is object-based in the sense that almost everything in JavaScript is an object.
  - Objects can be created dynamically without requiring a class.
- JavaScript is also object-oriented because it supports OOP principles: inheritance, encapsulation, polymorphism and Abstraction.

Characteristics of JavaScript(cont..)

- **JavaScript is event driven**
  - Events can trigger functions and World Wide Web is based upon an event-driven model. For example, whenever you click an item on a Web page, an event occurs. On a Web page, the user is in control and can click or not click, move the mouse or not move the mouse, or change the URL.
- **JavaScript is platform independent**
  - JavaScript programs are designed to run within HTML documents, they are not tied to any specific hardware platform or operating system. However, JavaScript programs are tied to a specific **user agent** (browsers). Theoretically, you can implement the same JavaScript program on any current user agent, such as Netscape Navigator, or Internet Explorer or others.

Characteristics of JavaScript(cont..)

- **JavaScript Enables quick development**
  - JavaScript does not require time-consuming compilation, scripts can be developed quickly.
- **JavaScript is Relatively easy to learn**
  - JavaScript does not have the complex syntax and rules associated with Java. Even if you do not know any other programming language, learning JavaScript will not be difficult**.**

JavaScript Syntax

- A JavaScript consists of JavaScript statements that are placed within the <script>... </script> HTML tags in a web page.
- The <script> tag alert the browser program to begin interpreting all the text between these tags as a script.
- So simple syntax of your JavaScript will be as follows <script ...> JavaScript code </script>
- So your JavaScript segment will look like:

```
<script language="javascript" type="text/javascript">
 JavaScript code
        </script>
```

JavaScript Syntax (cont.)

- The script tag takes two important attributes:
- **language**: This attribute specifies what scripting language you are using. Typically, its value will be javascript.
- **Type**: This attribute is what is now recommended to indicate the scripting language in use and its value should be set to “ “text/javascript”.

JavaScript Basics

- ✓ **Commenting out script in html**
  - Comments will not be executed by JavaScript.
  - Comments can be added to explain the JavaScript, or to make the code more readable.
  - Single line comments start with //.
  - Multi line comments start with /* and end with */.
  - JavaScript also recognizes the HTML comment opening sequence <!--. JavaScript treats this as a single-line comment.
  - The HTML comment closing sequence --> is not recognized by JavaScript so it should be written as //-->

- ✓ **JavaScript Statements**
  - Are a command to tell the browser what to do.
  - Statement end with semicolon

  e.g The statement that tells the browser to write "Hello World" to the web page is:

```
document.write("Hello World");
```

JavaScript Basics (Cont.)

- ✓ Semicolons are Optional:
  - Simple statements in JavaScript are generally followed by a semicolon character, just as they are in C, C++, and Java.
  - JavaScript, however, allows you to omit this semicolon if your statements are each placed on a separate line.
  - For example, the following code could be written without semicolons

```
<script language="javascript" type="text/javascript">
        var1 = 10
        var2 = 20
</script>
```

  - But when formatted in a single line as follows, the semicolons are required:

```
<script language="javascript" type="text/javascript">
 var1 = 10; var2 = 20;
</script>
```

JavaScript Basics (Cont.)

Variables

- Are container for information you want to store.
- Untyped!
- Declare a variable: you can create a variable with or without the **var or let** statement

  var strname = some value

  let strname = some value

  strname = some value
- Storing a value in a variable is called variable initialization
- You can do variable initialization at the time of variable creation or later point in time when you need that variable as follows:

```
var hello="Hi Everybody"; or
var hello;
hello=1;
```

JavaScript Basics(cont…)

- ✓ **Rule for variable name**
  - Variable name are case sensitive
  - They must begin with a letter or the underscore character
  - You should not use any of the JavaScript reserved keyword as variable name.
- ✓ **Literals**
  - In JavaScript literals can be of the following types
  - Numbers **17 , 123.45**
  - Strings; **"Hello Everybody"**
  - Boolean: **true , false**
  - Arrays: **[1,"Hi",17.234]** Arrays can hold anything!

JavaScript Basics(cont…)

- **Arrays**
  - Arrays are JavaScript Objects.
  - The array object is used to store a set of values in a single variable name
  - Arrays can grow dynamically – just add new elements at the end.
  - Arrays can have holes, elements that have no value.
  - Array elements can be anything (numbers, strings etc..)

Creating Array Objects

1. With the new operator and a size:

```
var x = new Array(10);
```

2. With the new operator and an initial set of element values:

```
var y = new Array(18,"hi",22);
```

3. Assignment of an array literal

```
var x = [1,0,2];
```

4.to access the elements of an array

```
arrayName[elementIndex]
```

example

```
<HTML>
      <HEAD>
  <SCRIPT>
anArray = new Array();
anArray[0] = "apple";
anArray[2] = "Orange";
anArray[9] = 1999;
//display all elements of this array
for (i = 0; i < anArray.length; i++){
   document.write("anArray at "+i+" = "+anArray[i]
   +"<BR>");}
</script></head><body></body></HTML>
```

Output

anArray at 0 = apple
anArray at 1 = undefined
anArray at 2 = Orange
anArray at 3 = undefined
anArray at 4 = undefined
anArray at 5 = undefined
anArray at 6 = undefined
anArray at 7 = undefined
anArray at 8 = undefined
anArray at 9 = 1999

JavaScript Basics(cont…)

- **JavaScript operators**
  - Arithmetic, comparison, assignment, logical (pretty much just like C++).
- **Arithmetic operators**

  + - * / % ++ --

  - The + operator is used for addition (if both operands are numbers) or The + operator means string concatenation (if either one of the operands is not a number).
  - Example

```
x=5+5;
document.write(x);        Out Put is 10
x="5"+"5";
document.write(x);        Out Put is 55
x="5"+
5;
document.write(x);        Out Put is 55
```

- **Assignment operators**

  = += -= *= /* %=
- **Comparison operators**

  == != > < <= >=
- **Logical operators**

  && || !

JavaScript Basics(cont…)

- **Conditional Statements**

  JavaScript supports the following conditional statements if..., if...else , if...else if and switch statement
- If you want to execute some code if a condition is true use **If statement**

  Syntax

```
if(condition)
{code to execute if the condition is true
}
```

  Example

```
<script type="text/javascript">
var age = 20;
if( age > 18 )
{
 document.write("<b>Qualifies for driving</b>");
}
</script>
```

JavaScript Basics(cont…)

- If you want to execute code if a condition is true and another code if the condition is false use **if…else statement.**

Syntax

```
If(condition)
{code to be execute if the condition is true
}
Else
{Code to  be executed if the condition is false
}
```

Example

```
<script type="text/javascript">
var age = 15;
if( age > 18 ){
   document.write("<b>Qualifies for driving</b>");
}else{
   document.write("<b>Does not qualify for driving</b>");
}
</script>
```

JavaScript Basics(cont…)

- The if...else if... statement is the one level advance form of control statement that allows JavaScript to make correct decision out of several conditions.

Syntax:

```
if (expression 1){
  Statement(s) to be executed if expression 1 is true
}else if (expression 2){
  Statement(s) to be executed if expression 2 is true
}else{
  Statement(s) to be executed if no expression is true
}
```

Example:

```
<script type="text/javascript">
var book = "maths";
if( book == "history" )
{   document.write("<b>History Book</b>"); }
else if( book == "maths" )
{   document.write("<b>Maths Book</b>");
}
else{   document.write("<b>Unknown Book</b>"); }
</script>
```

JavaScript Basics(cont…)

- You can use multiple if...else if statements, as in the previous , to perform a multi way branch.
- However, this is not always the best solution, especially when all of the branches depend on the value of a single variable.
- If you want to select one of many blocks of code to be executed
- You can use a switch statement which handles this situation, and it does so more efficiently than repeated if...else if statements.

Syntax

```
Switch (expression)
{
Case lable1:
Code to be execute if the expression =lable1
Brake
Case lable 2:
Code to be execute if the expression =lable2
Brake
Default:
Code to be execute if the expression is different from both lable1 and lable1
}
```

JavaScript Basics(cont…)

Example:

```
<script type="text/javascript">
var grade='A';
switch (grade){
case 'A': document.write("Good job<br />");
          break;
case 'B': document.write("Pretty good<br />");
          break;
case 'C': document.write("Passed<br />");
          break;
case 'D': document.write("Not so good<br />");
          break;
case 'F': document.write("Failed<br />");
          break;
default:  document.write("Unknown grade<br />")}
document.write("Exiting switch block");
</script>
```

Loop statements

- Very often when we want to run some block of code a number of time we use looping statements.
- In the java script we have the following looping statements.
  - Do…while, while, for.
- **While :-The While** statements will execute a block of code wile a condition is true.

Syntax

```
While (condition)
{
Code to be executed
}
```

Loop statements(cont…)

- **Do…while statement** will execute a block of code once, and then it will repeat the loop while the condition is true
  - Syntax

    Do {
    Code to be execute
    }
    While(condition)
- **For statement** execute a block of code a specific number of times
  - Syntax

  For (initialization; condition; increment)
  {
  code to execute
  }
- **Example:** The following JavaScript code which display "JavaScript" 3 times

```
<script language="text/JavaScript">
                    var x=0;
for(x=0;x<3;x++){
                    document.write("<h1>JavaScript</h1>"+"<br>");
    }
</script>
```

JavaScript function

- A function is a reusable code-block that will be executed by ***an event or when the function is called.***
- function declaration

  Function myfunction(argument1, argument2, etc)

  {

  Some statements

  }
- Call a function

  myfunction(argument1, argument2, etc)

Example:

There is a button having the value "Message" and if u click it the function message() will be called.

<input type="button" value="Message" onClick="message()">

The function will display an alert which says Hello World

```
function message() {
alert("Hello World");
 }
```

JavaScript Popup Boxes

There are three basic popup boxes as follows,

1. Alert box

- An alert box is often used if you want to make sure information comes through to the user.
- User will have to click "OK" to proceed

```
alert("sometext");
```

2. Confirm box

- User will have to click either "OK" or "Cancel" to proceed
- A confirm box is often used if you want the user to verify or accept something.

```
confirm("sometext");
```

3. Prompt(on time) box

- User will have to click either "OK" or "Cancel" to proceed after entering an input value
- A prompt box is often used if you want the user to input a value before entering a page.

```
prompt("sometext","defaultvalue");
```

Embedding JavaScript into HTML

- JavaScript codes can be linked with HTML markup language in two ways,

1. External JavaScript files

- There are distinct advantages to linking external JavaScript (having **.js** file extension) to HTML
- External JavaScript code don't have <script> tag.
- The principal of separation of development layers encourages us to keep different aspects of a document apart, typically in separate, linked files. Doing so facilitates,
  - Cleaner development,
  - Faster downloads, and
  - To make the code much easier to modify together in a single file.

Syntax

```
<html>
<head>
<script language="JavaScript" type="text/javascript" src="myOwnJavaScript.js">
</script>
</head>
<body>
//some code
</body>
</html>
```

2. **JavaScript into HTML**

- Place it into an HTML document using the <SCRIPT> tag to the head or the body section (or both).
- Script in the head section will executed when called
- If you want to have a script run on some event, such as when a user clicks somewhere, then you will place that script in the head as follows:

```
<html> <head>
<script type="text/javascript">
function sayHello()
{
alert("Hello World")
}
 </script>
 </head>
 <body>
 <input type="button" onclick="sayHello()" value="Say Hello" />
 </body>
 </html>
```

2. **JavaScript into HTML(Cont.)**

- JavaScript in <body>...</body> section:
- Script in the body section will executed while the page loads.
- If you need a script to run as the page loads so that the script generates content in the page, the script goes in the <body> portion of the document. In this case you would not have any function defined using JavaScript:

```
<html>
 <head>
 </head>
 <body>
 <script type="text/javascript">
 document.write("Hello World")
 </script>
 <p>This is web page body </p>
 </body>
 </html>
```

Quiz ?

JavaScript that changes HTML styles dynamically using the style property

Events & Event Handlers

- Every element on a web page has certain events
- Events can trigger invocation of event handlers.
- Events can be,
  - A mouse click,
  - A web page or an image loading,
  - Mousing over a hot spot on the web page …, etc
- Some of pre defined event handler functions
  - onclick - Mouse clicks an object
  - ondblclick - Mouse double-clicks an object
  - onselect - Text is selected
  - onsubmit - The submit button is clicked
  - onunload - The user exits the page

Form Validation with JavaScript

- The process of checking that a form has been filled in correctly before it is processed.
- **A simple form with validation**

The form

- The first part of the form is the form tag:

```
<form name="contact_form" onsubmit="return validate_form ( );">
 <h1>Please Enter Your Name</h1>
<p>Your Name: <input type="text" name="contact_name"></p>
<p><input type="submit" name="send" value="Send Details"></p>
 </form>
```

- The form is given a name of "contact_form". This is so that we can reference the form by name from our JavaScript validation function.
- The form tag includes an **onsubmit** attribute to call our JavaScript validation function, **validate_form(),** when the submit type of button in the form is pressed.
- The function **validate_form()** will validate whether the user enters Name or not.

Validating text filed

- Checks to see whether the text filed Is fill or not
- Use built in function .value for validation

Example

```
<html>
<head>
<script>
function validateForm()
{
var x=document. contact_form.contact_name.value;
if (x==null || x=="")
  {
  alert("First name must be filled out");
  return false;
  }
}
</script>
</head>
```

```
<body>
<form name="myForm" onsubmit="return validateForm()“>
First name: <input type="text" name="fname">
<input type="submit" value="Submit">
</form>
</body>
</html>
```

Validating radio buttons

- Checks to see whether either of the radio buttons have been clicked.
- Mostly used for unique identification
- Use built in function .checked for validation

Example

```
<html>
<head>
<script>
function validation()
{
 if ( ( document.reg_form.dept[0].checked == false )  &&
    (document.reg_form.dept[1].checked == false) )
                    {      alert ( "Please choose your Gender:
Male or Female" );
                    valid = false;
                    }
                    }
</script>
</head>
```

```
<body >
<form name="reg_form">
<input type="radio" name="dept"
value="female">female
<input type="radio" name="dept"
value="male">male
<input type="submit" onclick="validation()"
value="register">
</form>
</body>
</html>
```

Validating drop-down lists

- Used to make sure that one of the elements from drop down list has been selected or not.
- The pre defined function .selctedIndex can get the selected element from the drop down list.

example

```
<html>
<head>
<script>
function validation()
{
if ( document.reg_form.dept.selectedIndex == 0 )
                        {       alert ( "Please select your departement." );
                                return false;
                        }
                        }
</script>
</head>
```

```
<body>
<form name="reg_form">
department<select name="dept">
<option>select your department here</option>
<option>computer science</option>
<option>information technology</option>
<option>information system</option>
</select>
<input type="button" value="register"
onclick="validation()">
</body>
</html>
```

JavaScript Object

- JavaScript has many pre determined objects
- Objects have attributes and methods.

Document property

- **Document.title:-display the title of the document**

Document Methods

- **document.write()** a print statement the output goes into the HTML document.
- **document.writeln()** adds a newline after printing.

  E.g **document.write("My title is" + document.title);**

The window Object

- Access to, and control of, a number of properties including position and size.
- Represents the current window.

  window.document.write("Hello World!");

The Math Object

- Access to mathematical functions and constants.
- Constants: **Math.PI**
- Methods:
  - **Math.abs(), Math.sin(),**
  - **Math.log(), Math.max(),**
  - **Math.pow(), Math.random(),**